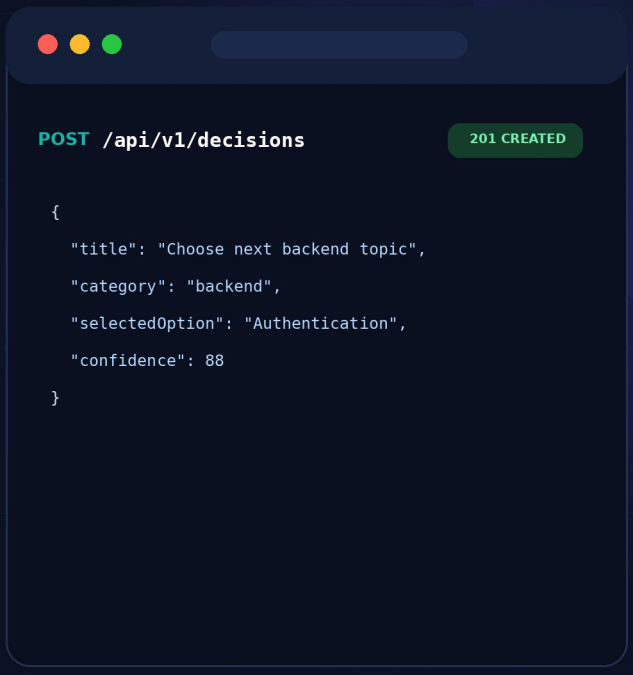
DL **DecodeLabs**
Full Stack Development Internship

PROJECT 2 / BACKEND API DEVELOPMENT

ClarityLoop Decision Journal API

A backend REST API for recording decisions, reviewing outcomes, filtering history and turning confidence into measurable insight.

Node.js | Express.js | REST API | JSON | Postman | Automated Testing



07	08	100%
API endpoints	automated tests	test pass rate

PREPARED BY
Mustafa Tahir
Full Stack Development Intern

SUBMITTED TO
DecodeLabs
Training & Mentorship Team

DATE
2026
Project 2 - Final Report

TABLE OF CONTENTS

Report Contents

Submission snapshot

Project: ClarityLoop Decision Journal API

Track: Backend API Development

Core stack: Node.js + Express.js + JSON + Postman

Verification: 8 automated tests, 8 passed, 0 failed

Executive Summary

ClarityLoop is a backend REST API built to turn decision-making into a measurable loop. Instead of storing only a final choice, the API records the decision context, options, selected choice, reasoning and confidence, then allows the outcome to be reviewed later. This creates a simple but meaningful backend workflow in which decisions can be revisited, filtered and analysed rather than forgotten.

The project was developed for DecodeLabs Project 2, whose core requirements are to create GET and POST endpoints, handle user input and responses, and validate basic data. The final implementation goes further by introducing reusable service logic, JSON persistence, structured error handling, filtering, outcome reviews, confidence-versus-outcome analytics, Postman testing and an automated regression suite.

07	08	100%	JSON
API endpoints	automated tests	pass rate	persistence

ClarityLoop separates request handling, validation, application logic and persistence into a clear backend flow.

1 Project Introduction

From API requirements to a decision-intelligence workflow.

Project 2 marks the transition from frontend interface work to the application's server-side brain. ClarityLoop was designed as a practical REST API that receives user input, validates it, applies application logic, persists the result and returns predictable JSON responses.

1.1 Problem and Purpose

A decision is often recorded as a final answer without preserving the reasoning behind it. That makes it difficult to learn from past judgement. ClarityLoop addresses this by recording the context, alternatives, selected option, reasoning and confidence at the moment a decision is made, then attaching an outcome review later.

1.2 Project Requirements

The DecodeLabs Project 2 brief requires a simple backend API with GET and POST endpoints, user-input handling, structured responses and basic validation. ClarityLoop satisfies each requirement directly while adding filtering, persistence, outcome reviews, analytics and automated testing.

1.3 Objectives

- → Build a clear REST API using Node.js and Express.js.
- → Handle JSON requests and return structured, predictable responses.
- → Validate decision and review data before it reaches persistence.
- → Use meaningful HTTP status codes for success and error conditions.
- → Store decision records in a lightweight JSON persistence layer.
- → Verify behaviour manually with Postman and automatically with repeatable tests.

Final outcome

A working backend service with seven documented endpoints, JSON persistence, server-side validation, decision filtering, review capture and confidence-versus-outcome insights.

The project remains intentionally lightweight so the next internship milestone can replace the file store with a relational database.

2 Technology Stack and Architecture

A small stack with clear separation of concerns.

Technology	Role	Evidence in ClarityLoop
Node.js	Server runtime	Runs backend JavaScript and test suite
Express.js	REST framework	Routes requests and structures responses
JSON	Data + persistence	Stores decisions and API payloads
Postman	Manual testing	Validates requests, status codes and responses
Node Test Runner	Automated testing	Runs eight repeatable regression tests
Git / GitHub	Version control	Stores source, evidence and documentation
Railway	Deployment	Hosts the Project 2 backend service

2.1 Backend Architecture

The codebase separates HTTP transport from reusable application logic. The route layer handles endpoints and status codes, while the service layer owns validation, filtering, file persistence and insight calculations. This makes the most important logic testable without requiring a live web server for every test.

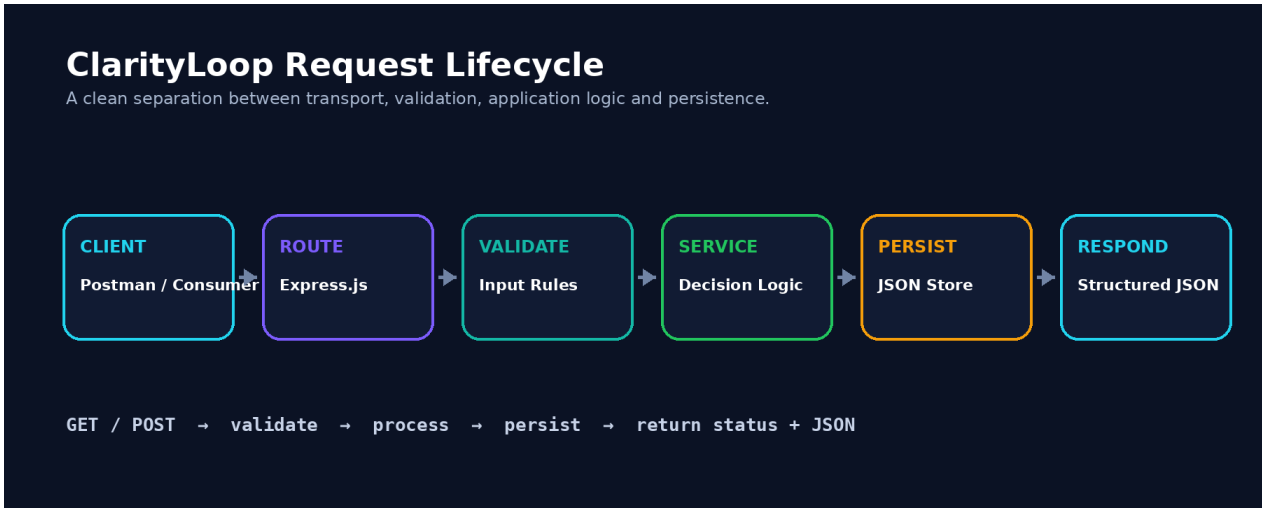


Figure 1. Request lifecycle and separation of concerns in the Project 2 backend.

3 REST API Design

Resource-oriented routes, predictable methods and structured responses.

Method	Endpoint	Purpose
GET	/	API metadata
GET	/api/health	Health check
GET	/api/v1/decisions	Retrieve / filter decisions
GET	/api/v1/decisions/:id	Retrieve one decision
POST	/api/v1/decisions	Create a decision
POST	/api/v1/decisions/:id/review	Add outcome review
GET	/api/v1/insights	Generate analytics

3.1 RESTful Naming

Routes are named around resources rather than actions. The API uses `/decisions` for the decision collection, path parameters for individual records and HTTP methods to express the operation. This avoids route names such as `/createDecision` and keeps the interface easier to understand.

3.2 Structured Responses

Successful creation returns HTTP 201, normal retrieval returns 200, invalid input returns 400 and missing resources return 404. Responses consistently expose a success flag and either data or a clear human-readable message.

Figure 2. Decision creation request illustrating the POST workflow and HTTP 201 response.

4 Validation and Error Handling

Protecting data quality before persistence.

Validation is applied before new information reaches the JSON store. Required text fields must contain meaningful content, confidence must be an integer between 1 and 100, each decision must provide at least two options and the selected option must exist in the supplied list.

4.1 Review Validation

→ Outcome is restricted to positive, mixed or negative.

→ Outcome score must be an integer between 1 and 10.

→ Actual outcome is required.

→ Lesson learned is required.

4.2 HTTP Error Strategy

✓ 400 Bad Request - invalid or incomplete input.

✓ 404 Not Found - unknown decision or route.

✓ Clear validation messages - errors explain what must be fixed.

✓ Validation-before-write - malformed records are never persisted.

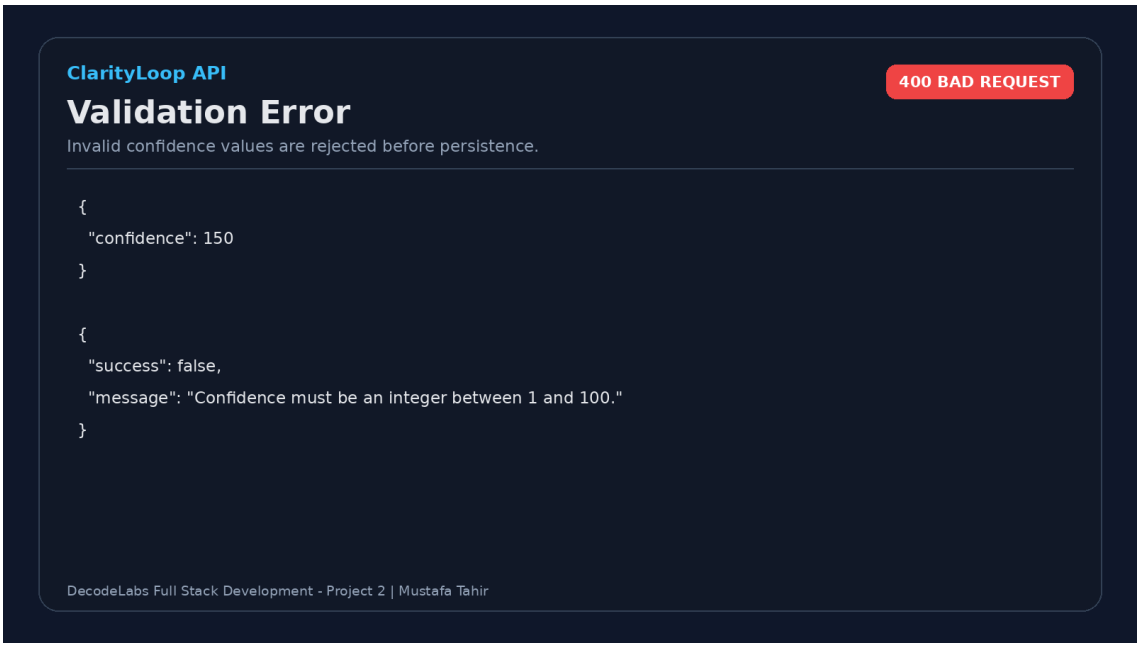


Figure 3. Example of server-side validation rejecting confidence outside the allowed range.

Why this matters

Backend validation protects the persistence layer from inconsistent records and gives API consumers predictable failure behaviour.

5 Filtering, Reviews and Insights

Moving from data storage to useful backend behaviour.

5.1 Filtering

The decisions endpoint accepts optional category and date query parameters. This keeps retrieval flexible without introducing duplicate endpoints for every filter combination.

Figure 4. Query-parameter filtering for decision history.

5.2 Outcome Review

A later review attaches the real outcome to an existing decision. It stores the outcome classification, score, actual result, lesson learned and review timestamp so the original reasoning can be compared with what happened.

5.3 Confidence-versus-Outcome Analytics

The insights endpoint calculates reviewed-decision count, average confidence, average outcome score, a normalized confidence-outcome gap and positive/mixed/negative outcome distribution.

Interpretation

A positive confidence-outcome gap indicates that average confidence was higher than the normalized outcome score, which can act as a simple overconfidence signal. A negative gap indicates stronger outcomes than initial confidence.

6 Testing and Verification

Manual endpoint checks backed by automated regression tests.

Project 2 combines manual API verification in Postman with an automated service-level test suite. Postman is useful for inspecting request bodies, status codes and response payloads, while automated tests provide a fast repeatable check of validation, filtering and analytics logic.

Test ID	Verification	Result
P2-T01	Valid decision accepted	PASS
P2-T02	Confidence above 100 rejected	PASS
P2-T03	Fewer than two options rejected	PASS
P2-T04	Invalid selected option rejected	PASS
P2-T05	Valid outcome review accepted	PASS
P2-T06	Invalid outcome score rejected	PASS
P2-T07	Category filtering works	PASS
P2-T08	Insights calculation works	PASS

Figure 5. Automated regression suite - eight tests executed, eight passed, zero failed.

8 tests / 8 passed / 0 failed

7 Persistence and Deployment

Keeping Project 2 lightweight while preparing for the database milestone.

7.1 JSON Persistence

Project 2 intentionally stores decision records in a local JSON file. This choice keeps the backend focused on API fundamentals while still demonstrating stateful behaviour across requests. The persistence layer can be read, updated and written without introducing database configuration before it is required by the internship sequence.

```
data/
└── decisions.json

POST /api/v1/decisions
        ↓
validate → append record → write JSON
```

7.2 Deployment

The Project 2 backend was deployed as a Node.js service using Railway. Deployment demonstrates that the API can run beyond the local development environment and expose its health and application endpoints over the web.

7.3 Persistence Limitation

> **Important engineering note**
>
> File-based persistence is suitable for this learning milestone but local files may be ephemeral on some cloud platforms. Project 3 replaces the JSON store with Microsoft SQL Server and relational database rules.

7.4 Progression to Project 3

Project 2 establishes the API layer. Project 3 preserves the backend concepts and upgrades persistence to a relational schema with primary keys, foreign keys, constraints, transactions and database-backed CRUD operations.

8 Challenges, Solutions and Learning Outcomes

What changed from 'it works' to a maintainable backend.

Challenge	Solution	Learning
Route logic becoming crowded	Moved validation and analysis into a service module	Separation of concerns improves maintainability
Invalid records reaching storage	Validated every decision and review before writes	Backend validation protects data quality
Testing analytics reliably	Kept calculations deterministic and service-level	Pure logic is easier to regression-test
Choosing persistence for scope	Used JSON instead of adding a database early	Technology choices should match the milestone

8.1 Skills Strengthened

✓ Node.js and Express.js backend development	✓ RESTful resource naming and HTTP methods
✓ JSON request / response handling	✓ Input validation and HTTP status codes
✓ File-based persistence	✓ Postman endpoint verification
✓ Automated tests with Node.js	✓ Technical documentation and deployment workflow

8.2 What Project 2 Proved

The project demonstrates that a backend is more than a collection of routes. Reliable API behaviour depends on validation, clear status codes, predictable response shapes, testable application logic and persistence choices that fit the stage of the system.

9 Conclusion and Submission Readiness

A complete backend milestone ready for review.

ClarityLoop successfully meets the DecodeLabs Project 2 backend-development objectives and presents them as a coherent product rather than a set of isolated endpoints. The final API accepts and validates decision data, stores records, retrieves and filters history, captures outcome reviews and generates simple confidence-versus-outcome analytics.

The submission also demonstrates software-engineering discipline through reusable service logic, structured error responses, Postman verification, an eight-test automated suite and clear documentation. This creates a strong bridge into Project 3, where the same backend concepts are extended with relational SQL Server persistence.

9.1 Submission Package

- ✓ Working Node.js and Express.js backend source.
- ✓ JSON persistence with sample decision records.
- ✓ Seven documented API endpoints.
- ✓ Postman collection for core workflows.
- ✓ Eight automated tests with a 100% pass rate.
- ✓ Architecture, API, testing and deployment documentation.
- ✓ Professional evidence images and final project report.

PROJECT 2 / COMPLETED / SUBMISSION-READY

ClarityLoop Decision Journal API • Mustafa Tahir • DecodeLabs • 2026